

CUDA Python Laboratory

Completed experimental task sheet

Applied Massive Parallelism: Thread Geometries and Hardware Latency Pitfalls

Student name	Student ID	Session / date	Assigned GPU ID
Ainabek Aisultan	230103254	8 October 2026	0

All empirical values below are transcribed from my supplied Colab console output. Explanations and calculations are based on those measurements.

1. Environment provisioning and hardware setup

Mandatory telemetry field	Recorded value
NVIDIA driver version	580.82.07
CUDA version supported by driver	13.0
Allocated GPU name	Tesla T4
Total available VRAM	15360 MiB
Compute capability	7.5
Maximum threads per block	1024
Multiprocessor count (SMs)	40
Maximum block dimensions (X, Y, Z)	1024, 1024, 64
Warp size	32

The nvidia-smi output identifies device 0 at bus address 00000000:00:04.0. At inspection, memory usage was 105 MiB out of 15360 MiB. CUDA 13.0 is the driver-supported version reported by nvidia-smi.

2. Vector doubling: empirical benchmark table

Array size N	CPU (ms)	GPU total (ms)	Kernel only (ms)	Winner
100	0.051173	1.233448	0.097351	CPU
1,000	0.607945	0.944415	0.100611	CPU
10,000	4.995591	0.727034	0.104109	GPU
100,000	59.788660	1.082221	0.106156	GPU
1,000,000	498.785479	5.632017	0.147668	GPU
5,000,000	N/A	15.121944	0.305526	N/A
10,000,000	N/A	28.356628	0.519577	N/A

CPU implementation: pure Python loop. GPU implementation: guarded 1D CUDA kernel, 256 threads per block. Random seed: 230103254. GPU output passed `np.allclose(h_out, h_in * 2)`. CPU timing was skipped for N greater than 1,000,000, as permitted by the manual; no winner is inferred for those rows.

Benchmark analysis and thread geometry

2. Analysis questions

Q1. What is the empirical crossover point N^* on your machine?

My observed crossover point is $N^* = 10,000$, the smallest tested array size for which GPU total time was lower than CPU time. At $N = 1,000$, the CPU took 0.607945 ms and the GPU took 0.944415 ms. At $N = 10,000$, the CPU took 4.995591 ms and the GPU took 0.727034 ms, giving approximately 6.87 times lower elapsed time for the GPU. The measurements bracket the transition between these two tested sizes; they do not locate the exact threshold within that interval.

Q2. At $N = 100$, why is GPU total time dramatically slower than CPU time?

The CPU needs only 0.051173 ms for 100 elements. The GPU total is 1.233448 ms, while the reported kernel-only measurement is 0.097351 ms. This small workload does not provide enough computation to offset host-to-device and device-to-host transfers, device allocation, and CUDA launch/runtime overhead. The GPU total is approximately 24.10 times the CPU time.

The difference is $1.233448 - 0.097351 = 1.136097$ ms, or approximately **92.11% of GPU total time**. My result therefore supports overhead dominating execution, but does not support the question's literal 95%+ figure. The two timings are separate runs, so this subtraction is an approximate overhead comparison. The reported kernel-only value also includes host launch overhead and `cuda.synchronize()`; it is not a pure device-event kernel measurement.

3.1 Missing-guard disaster experiment

Launch calculation: $\text{ceil}(1000 / 256) = 4$ blocks; $4 \times 256 = 1024$ threads. Threads with global IDs 1000 through 1023 are out of bounds: 24 extra threads.

Exact observed console behavior:

```
Launching 1024 threads for 1000 elements...
Execution completed.
Elements: 1000
Last element: 999.0
```

No CUDA exception or Python `IndexError` was displayed. A `NumbaPerformanceWarning` reported low occupancy for grid size 4. Successful completion does not make the kernel safe: the missing guard permits writes beyond the array. Such GPU writes can silently affect adjacent allocated memory or produce an asynchronous illegal-address error. My output does not establish whether adjacent memory was corrupted. The correct kernel must check `idx < d_arr.shape[0]` before writing.

3.2 Spatial thread coordinate inspector: $N = 37$

Total blocks	Threads launched	Active threads	Idle guarded threads
5	40	37	3

Ceiling division: $(37 + 8 - 1) // 8 = 5$ blocks. Global ID = block ID $\times$ 8 + thread ID. The final block covers global IDs 32 through 39.

Global ID	Block ID	Thread ID	Status
32	4	0	ACTIVE
33	4	1	ACTIVE
34	4	2	ACTIVE
35	4	3	ACTIVE
36	4	4	ACTIVE
37	4	5	IDLE (GUARDED)
38	4	6	IDLE (GUARDED)
39	4	7	IDLE (GUARDED)

Race condition and atomic reduction

4.1 Mandatory experimental proof

Input: 50,000 float32 values, each equal to 1.0. Expected sum: **50,000.0**. Configuration: 196 blocks of 256 threads, with a boundary guard. The scalar accumulator was reset to zero before each trial.

Trial	Target sum	Naive output	Lost updates	Time (ms)
1	50,000.0	2.0	49,998.0	0.2507
2	50,000.0	2.0	49,998.0	0.2140
3	50,000.0	2.0	49,998.0	0.1824
4	50,000.0	2.0	49,998.0	0.2079
5	50,000.0	2.0	49,998.0	0.1744

Deviation calculation: lost updates = 50,000.0 - 2.0 = 49,998.0 in every trial. This is 99.996% of the target sum.

The operation `d_total[0] += d_arr[idx]` performs an unprotected read-modify-write. Many threads can read the same old value and overwrite each other's results, losing additions. All five observed outputs happened to equal 2.0; repeated identical values do not prove the absence of a race condition or guarantee that a future run will match.

Correct atomic summation

Atomic trial	Recorded sum	Time (ms)
1	50,000.0	0.4825
2	50,000.0	0.3766
3	50,000.0	0.2923
4	50,000.0	1.2310
5	50,000.0	0.3467

Atomic sum output: 50,000.0. **Does it equal exactly 50,000.0?** Yes, in all five recorded trials.

Performance penalty comparison

Measurement	Recorded mean time
Naive kernel	0.20587519998116477 ms
Atomic kernel	0.5458115999886104 ms

The atomic mean is approximately **2.65 times** the naive mean, an increase of about **165.12%**. Atomic Trial 4 was slower than the other trials and increases the mean, so this ratio describes the recorded experiment rather than a universal penalty.

Why do atomic operations incur a performance penalty? Every thread updates the same accumulator address. `cuda.atomic.add` prevents lost updates by making each addition indivisible, but competing updates to that address must be serialized. This contention reduces parallel throughput. The naive kernel avoids that protection but returns an incorrect sum. Both recorded timings also include host launch and synchronization overhead.

2D geometry and final verification

5. 2D grid image / matrix processing

The radial vignette kernel processed a **1024 x 1024** float32 canvas using 2D blocks of **16 x 16** threads. Each thread computed its pixel coordinates with `cuda.grid(2)`, checked the image boundaries, and wrote its intensity to `d_canvas[y, x]`.

Mandatory verification field	Recorded value
Blocks in X	64
Blocks in Y	64
Total blocks	4096
Pixel value at center (512, 512)	1.0
Pixel value at outer corner (0, 0)	0.0
Generated matrix shape	(1024, 1024)

Geometry calculation: $\text{ceil}(1024 / 16) = 64$ blocks in each direction; $64 \times 64 = 4096$ blocks. Each block contains 256 threads, giving 1,048,576 launched threads, exactly one per pixel.

Pixel calculation: $\text{intensity} = \max(0, 1 - \text{distance} / \text{max_distance})$, with center (512, 512). At the center, distance = 0 and intensity = 1.0. At (0, 0), distance = $\sqrt{512^2 + 512^2} = \text{max_distance}$, so intensity = 0.0. These calculations agree with the recorded console output.

6. Final verification checksum

Verification input / output	Value
Student ID used in report and benchmark seed	230103254
Empirical crossover N*	10000
Reported GPU name	Tesla T4
Unique 16-character verification token	272FE4A06FB1AFDA

Final submission token: 272FE4A06FB1AFDA

The token above is copied exactly from the supplied final Colab output. The official function hashes the student-ID string, GPU name, the first 10 x 10 canvas values, and the crossover value, then returns the first 16 hexadecimal characters in uppercase. The supplied excerpt does not show the `STUDENT_ID` assignment or function definition, so the token is transcribed rather than independently regenerated.

Final submission checklist

The experimental outputs and checksum are recorded in this task sheet. Complete the following notebook and submission steps in Colab:

- [] Save the notebook in Google Drive as `CUDA_Lab01_230103254.ipynb`.
- [] Confirm that all code cells were executed sequentially and their outputs remain visible.
- [] Export via File > Download > Download .ipynb and upload the notebook to the course portal.
- [] Submit this completed PDF task sheet to the instructor.

Submission actions are left unchecked because notebook saving, export, and portal upload are not confirmed by the supplied console text.